HTML & CSS PRACTICAL RECORD

Documentation of Three Web Design Experiments

1. Fitness Center and Gym Website
2. College Fest Website
3. Electronic Store Website

Prepared from the six HTML/CSS files supplied for this practical examination.

Each experiment includes: Introduction • Aim • Explanation of HTML elements • Explanation of CSS properties • Expected output • Procedure • Result • Viva preparation

Study note: This documentation describes the code as supplied. Any observed errors or limitations are identified separately rather than silently changing the original programs.

How to use this document

The three experiments are documented independently. For each one, first learn the aim and introduction, then understand the HTML structure, then the CSS rules, and finally practise explaining the browser output. The filenames are mapped as follows:

Experiment	HTML file	CSS file
1. Fitness Center / Gym	main gym.html	main.css
2. College Fest	college fest.html	style.css
3. Electronic Store	main.html	main(1).css

Important: The CSS file named *main.css* belongs to the gym page in the supplied set. The electronic-store page also links to a file named *main.css*, while the separately supplied CSS file is named *main(1).css*. When running the electronic-store page, rename or copy *main(1).css* to *main.css*, or update the HTML link to match the actual filename.

Common concepts used in all three experiments

• **HTML:** HyperText Markup Language describes the structure and content of a web page.

• **CSS:** Cascading Style Sheets controls presentation such as colours, spacing, alignment, typography and layout.

• **Element:** An HTML element usually consists of an opening tag, content and a closing tag, for example `<p>Hello</p>`.

• **Attribute:** Extra information written inside an opening tag, for example `class='ss1'` or `href='#events'`.

• **Class selector:** A CSS selector beginning with a dot, such as `.ss1`, applies styles to elements carrying that class.

• **ID selector:** A CSS selector beginning with # targets an element with a matching id, such as `#register`.

• **External stylesheet:** A separate .css file connected using the HTML `<link>` element.

• **Browser output:** The visual page produced when the browser interprets the HTML and CSS.

EXPERIMENT 1 — FITNESS CENTER AND GYM WEBSITE

1. Introduction

This experiment creates a simple Fitness Center and Gym website using HTML and CSS. The page introduces a fitness centre, lists workout programmes, displays membership plans and a trainer, and provides a basic registration area. HTML is used to arrange the information, while CSS styles the page with background colours, text alignment, font settings and button formatting.

2. Aim

To design a basic gym website using HTML elements and an external CSS stylesheet, and to display workout programmes, membership plans, trainer information and a registration interface.

3. Files used

File	Purpose
main gym.html	Contains the gym website content and form controls.
main.css	Contains the page styling rules.

4. Explanation of HTML elements

Element / code	Explanation and role in this experiment
<!DOCTYPE html>	Declares that the document uses HTML5. It helps the browser render the page in standards mode.
<html>	Root element that contains the complete HTML document.
<head>	Contains page information and linked resources; it is not the main visible page content.
<title>Gym Website</title>	Sets the title shown in the browser tab.
<meta charset='UTF-8'>	Specifies UTF-8 character encoding, allowing symbols and many language characters to display correctly.
<link rel='stylesheet' href='main.css'>	Connects the external CSS file named main.css to the HTML page.
<body>	Contains all visible webpage content.
	Creates a division/container. The class name ss can be used to apply CSS styles to this container.
<h1>	Displays the main heading: Fitness Center and Gym Website.
<p>	Displays paragraph text. Here it is used for the welcome message, programme names, prices and trainer details.
<h2>	Displays section headings such as Workout Programs, Membership Plans, Trainer and Registration.
<input type='text'>	Creates a single-line text field for the visitor's name. The placeholder gives a hint inside the field.
<input type='email'>	Creates an email input field. Browsers can perform basic email-format validation when it is used in a form.
 	Inserts a line break. The two consecutive br elements create vertical separation between controls.

Element / code	Explanation and role in this experiment
<button>Join Now</button>	Creates a clickable button. In the supplied code it is not connected to a form submission or JavaScript action.
, </body>, </html>	Close the container, visible document body and root HTML document respectively.

5. Explanation of CSS properties

Selector / property	Meaning and visible effect
body	Targets the complete page body.
background-color: white	Sets white as the body's background colour. A later body rule also sets a background image.
text-align: center	Centres inline content and text within the body.
font-family: Arial	Uses the Arial typeface, or a suitable fallback if Arial is unavailable.
background-image: url(...)	Requests an image as the body background. The first supplied URL is a Google image-results URL rather than a direct image file URL, so it may not display reliably.
h1 { background-color: black; color: white; padding: 15px; }	Gives the main heading a black background, white text and 15 pixels of inner spacing.
h2 { color: red; }	Makes second-level headings red.
button { background-color: orange; padding: 10px; }	Makes the button orange and adds 10 pixels of inner spacing.
ss { background-image: url(...); }	Targets an HTML element named ss. It does not target class='ss'; a class selector would need to be .ss.
body { background-image: url('d.jpg'); background-size: cover; background-repeat: no-repeat; }	A second body rule sets d.jpg as the background, scales it to cover the available area and prevents tiling. Because this rule comes later, it overrides the earlier body background-image declaration.
p { color: blue; }	Makes paragraph text blue.

6. Expected browser output

When the HTML page is opened in a browser, the user should see a centred gym webpage. The top contains the main heading “Fitness Center and Gym Website” in white text on a black background. A welcome message appears below it. The page then shows the Workout Programs section with Yoga, Cardio and Weight Training; the Membership Plans section with Basic (■500), Standard (■1000) and Premium (■1500); the Trainer section naming shanmukh; and the Registration section with name and email fields and an orange Join Now button. Paragraph text is styled blue and section headings are red.

The exact background image depends on whether d.jpg exists in the correct folder. If it is missing, the browser will not show that image. The CSS rule for .ss is not present in the supplied file; the written selector is ss without a dot.

7. Procedure / steps to execute

• Create a folder for the gym experiment.

• Save the supplied HTML code as **main gym.html** and the CSS code as **main.css** in the same folder.

• Keep the background image file named **d.jpg** in that folder if the second background rule is to display it.

• Check that the HTML link element refers to the correct CSS filename.

• Open main gym.html in a web browser.

• Observe the heading, programme list, membership prices, trainer details, input fields and button.

8. Result

Thus, a basic Fitness Center and Gym website was created using HTML and CSS. It displays workout programmes, membership plans, trainer information and a simple registration interface.

EXPERIMENT 2 — COLLEGE FEST WEBSITE

1. Introduction

This experiment designs a College Fest 2026 webpage. It presents the fest title and slogan, navigation links, technical and cultural events, workshops, an event schedule, fest rules, a registration form and a footer. HTML semantic elements such as header, nav, section and footer organise the content. CSS adds page colours, spacing and typography.

2. Aim

To create a College Fest website using HTML and CSS that displays event information, schedule, rules and a student registration form.

3. Files used

File	Purpose
college fest.html	Contains the college fest content, navigation, schedule and registration form.
style.css	Contains the body, header, navigation and body-class styles.

4. Explanation of HTML elements

Element / code	Explanation and role
<!DOCTYPE html>	Declares an HTML5 document.
<html>, <head>, <body>	The root document, metadata/resource area and visible content area.
<title>College Fest</title>	Sets the browser tab title to College Fest.
<link rel='stylesheet' href='style.css'>	Links the external stylesheet style.css.
<body class='ss1'>	Starts the visible page and assigns the class ss1. The CSS file uses .ss1 to give this body an orange background.
<header>	Semantic container for introductory content. Here it contains the fest heading and slogan.
COLLEGE FEST 2026	The primary heading identifying the event.
<p>	Displays the slogan and individual event names, rules and footer text.
<nav>	Semantic navigation area. It contains links to sections of the same page.
<a href='#home'> etc.	Creates clickable links. A fragment link navigates to an element whose id matches the fragment, for example #events should lead to id='events'.
<section id='events'>	Groups the event information and provides the target id events.
<h2> and <h3>	Represent section and subsection headings. h2 introduces a major section; h3 identifies Technical Events, Cultural Events and Workshops.
	Centres its contents. This is an older HTML element; modern pages normally use CSS text-align instead.
<table>	Creates a table for the event schedule.
<tr>	Creates a table row.
<th>	Creates a header cell, normally bold and centred by browsers.
<td>	Creates a normal data cell containing a time, event or coordinator.

Element / code	Explanation and role
<form>	Groups controls used to collect registration information.
<label>	Provides a text label describing an input field.
<input type='text' required>	Collects a student name. required asks the browser to require a value before a normal form submission.
<input type='email' required>	Collects an email address and enables basic browser email validation.
<button type='submit'>	Submits the form according to the form's configured submission behaviour. No action or backend endpoint is specified here.
<footer>	Semantic footer area. It contains the copyright-style fest line.
</...>	Closing tags end the elements and define the nesting structure.

5. Explanation of CSS properties

Selector / property	Meaning and visible effect
body	Targets the body element.
font-family: Arial, sans-serif	Uses Arial; if unavailable, the browser chooses a sans-serif font.
margin: 0	Removes the browser's default outer margin around the body.
background-color: cornsilk	Sets a pale yellow/cream background colour.
header	Targets the semantic header element.
background-color: #4b0082	Sets the header background to indigo/purple.
color: yellow	Sets header text to yellow.
text-align: center	Centres the header's text.
padding: 20px	Adds 20 pixels of inner spacing on all sides of the header.
nav	Targets the navigation bar.
background-color: green	Gives the navigation bar a green background.
text-align: center	Centres the navigation links.
padding: 12px	Adds inner spacing around navigation content.
.ss1 { background-color: orange; }	Selects the body element carrying class ss1 and sets its background to orange. This later class rule overrides the body's cornsilk background where it applies.

6. Expected browser output

The browser displays a College Fest 2026 page. At the top is a purple header with yellow, centred text and the slogan “Celebrate Talent, Technology and Culture!”. Beneath it is a green navigation bar with Home, Events, Schedule and Register links. The main content presents technical events (Coding Competition, Web Design and Quiz Competition), cultural events (Singing, Dancing and Drama), and workshops (Artificial Intelligence, Web Development and Robotics).

The schedule table contains columns Time, Event and Coordinator, with rows for Inauguration at 9:00 AM (Ravi), Coding Competition at 10:00 AM (Priya), Cultural Programs at 1:00 PM (Rahul), and Prize Distribution at 3:00 PM (Event Team). The rules are listed below, followed by a registration form for student name and email and a footer reading “2026 College Fest | All Rights Reserved”.

7. Procedure / steps to execute

• Save the HTML code as **college fest.html** and the CSS code as **style.css** in the same folder.

• Ensure the link in the head points to style.css.

• Open college fest.html in a browser.

• Check the header and navigation bar colours and alignment.

• Read the event categories and schedule table.

• Try the registration fields and observe the browser's required-field and email validation.

8. Result

Thus, a College Fest 2026 webpage was designed using HTML and CSS. It presents fest information, event categories, schedule, rules and a student registration form.

9. Important code observations (as supplied)

The HTML file contains Git command lines inside the head section immediately after the stylesheet link: echo, git init, git add, git commit, git branch, git remote and git push. These are terminal commands, not HTML elements. They should be removed from the HTML file and run separately in a command-line terminal only if the intention is to upload the project to GitHub.

The navigation includes #home and #schedule, but the supplied HTML does not show elements with id='home' or id='schedule'. The Events and Register targets do exist. Therefore, those two navigation links may not jump to the intended sections. The supplied markup also contains a malformed closing heading tag written as <h/2> in the fest slogan area. It should be a normal closing tag </h2>.

EXPERIMENT 3 — ELECTRONIC STORE WEBSITE

1. Introduction

This experiment creates the home page of an Electronic Store website. The page displays a store heading and logo, links to product-category pages, a welcome description, a collection of products with prices and images, and store contact details. HTML structures the store content and CSS styles the page and product layout.

2. Aim

To design an electronic store webpage using HTML and CSS, including category navigation, product images, product names, prices and store address/contact information.

3. Files used

File	Purpose
main.html	Contains the electronic store page and product information.
main(1).css	The supplied CSS file containing class and product-layout rules. The HTML currently links to main.css, so the filename must be made consistent.

4. Explanation of HTML elements

Element / code	Explanation and role
<!DOCTYPE html>	Declares an HTML5 document.
<html lang='en'>	Root element; lang='en' identifies English as the document language.
<head>	Contains the title, character encoding, stylesheet link and internal style block.
<title>electronic store website</title>	Sets the browser tab title.
<meta charset='UTF-8'>	Sets UTF-8 character encoding.
<link rel='stylesheet' href='main.css'>	Links an external stylesheet. The supplied separate CSS file is named main(1).css, so the name/path needs to be reconciled.
<style> ... </style>	Defines internal CSS within the HTML document. Here it defines the .f111 class.
.f111 { padding-left: 15px; padding-right: 15px; }	Adds 15 pixels of left and right padding to elements carrying class f111.
<body class='box'>	Starts the visible content and assigns class box, which is styled in the supplied CSS.
	Centres the logo and main heading. This is an obsolete presentational element; CSS is the modern alternative.
<h1>	Displays a major heading. The page uses it for the store name and category-link block.
<img src='logo.jpg' alt='logo' width='250px;' height='250;'>	Displays the logo image. src identifies the image file; alt provides alternative text; width and height set its dimensions. The values should conventionally be numeric or valid CSS lengths without an extra semicolon inside the attribute.
	Creates a generic block container for grouping related content.
<a href='activity.html'> etc.	Creates links to laptop, mobile, watch and AC category pages. The linked files must exist for navigation to work.
 	Moves following content to a new line.
<h3>	Introduces the welcome message and descriptive text in this page.
<h2 class='as'>	Displays a second-level heading and applies the as class, which makes its text italic.

Element / code	Explanation and role
	Groups product cards into a flexbox row using the .products CSS rule.
	Groups one product's price, image and name.
<p class='price'>	Displays a product price and applies the price class for bold, larger text.
<img src='k19.jpg' alt='Samsung Fridge'> etc.	Displays a product image. src is the image filename and alt describes the image if it cannot be displayed or for assistive technology.
<footer>	Contains the store address and contact information at the bottom of the page.
	Marks important text and normally displays it in bold; this line also uses an inline colour style.
style='color:yellowgreen;'	An inline CSS declaration that makes the strong text yellow-green.
</...>	Closing tags finish the corresponding HTML elements.

5. Explanation of CSS properties

Selector / property	Meaning and visible effect
.box { background-color: antiquewhite; }	Targets the body with class box and gives it an antique-white background.
.f11 { padding-left: 15px; padding-right: 15px; }	Adds horizontal inner spacing to elements with class f11. This class is defined but is not used in the supplied HTML.
.as { font-style: italic; }	Displays text in italics for the element carrying class as.
.af { text-decoration-color: aqua; }	Sets the colour used for text decoration on elements with class af. It does not create an underline by itself, and the class is not used in the supplied HTML.
.sasi { padding-right: 500px; margin-right: 20px; }	Adds 500 pixels of right padding and 20 pixels of right margin to elements with class sasi. This class is not used in the supplied HTML.
.products { display: flex; }	Turns the product container into a flexbox layout, arranging its direct child product cards along a row by default.
gap: 100px	Creates 100 pixels of space between flex items.
align-items: flex-start	Aligns flex items to the start of the cross axis (the top edge for a normal row layout).
.product { text-align: center; }	Centres text inside each product card.
.product img { width: 250px; height: 250px; }	Sets each product image to 250 by 250 pixels. Images may be stretched if their original proportions differ.
.price { font-size: 18px; font-weight: bold; }	Displays prices at 18 pixels and in bold.

6. Expected browser output

The page begins with a centred “electronic store website” heading and a logo image. Below it is a block of category links for laptop, mobiles, watches and ACs, followed by a welcome heading and store description. The “Our Electronic Products” heading introduces two product rows. Each product card displays a price, image and product name.

Product text in supplied HTML	Displayed price
Samsung Fridge	■25,000
boAt Earbuds	■1,499
AC	■42,000
Redmi 15c prime	■32,000

Product text in supplied HTML	Displayed price
washing machine	■25,000
table fan	■4,000
samsung	■59,000
redmi tv	■25,000

At the bottom, the footer shows the address text “electronic stores .vijayawada” and a contact number beginning “9704858...”. The product cards are intended to appear in horizontal flex rows with gaps between them.

The actual images will appear only if logo.jpg and the referenced k19.jpg, k20.jpg, k17.jpg, k6.jpg, k21.jpg, k22.jpg, k3.jpg and k23.jpg files are available at the paths given. The category links also require activity.html, mobile.html, watch.html and ac.html to exist.

7. Procedure / steps to execute

• Save the HTML as **main.html**.

• Save the supplied electronic-store CSS as **main(1).css**.

• Make the stylesheet reference consistent: either rename main(1).css to main.css or change the HTML link href to main(1).css.

• Place the logo and product image files in the correct folder, preserving their filenames.

• Create or provide the linked category pages if those links are expected to work.

• Open main.html in a browser and inspect the title, logo, category links, product cards, prices and footer.

8. Result

Thus, an Electronic Store webpage was designed using HTML and CSS. It displays product categories, product images, prices, product names and store contact details.

9. Important code observations (as supplied)

The HTML links to main.css, but the uploaded CSS file is named main(1).css. Unless a file named main.css is also placed beside the HTML page, the external styles will not load. The internal .f111 rule is present in the HTML itself and therefore can still apply independently.

The external CSS selectors .f11, .af and .sasi are not used by any element in the supplied HTML. The .products, .product, .product img, .price, .box and .as selectors do match classes used in the HTML. The HTML also contains a malformed heading close tag written as <h/2>; the correct closing form is </h2>. Some image alt text does not match the product name (for example, the AC image has alt='boAt Earbuds').

LAST-MINUTE EXAM REVISION

Quick comparison of the three experiments

Experiment	Main HTML concepts	Main CSS concepts	What the output demonstrates
Gym website	Headings, paragraphs, div, text/email inputs, button, external link	Background image/colour, text alignment, font, padding, colours	Fitness services, membership plans, trainer and registration fields
College Fest	Header, nav, anchor links, section, table, form, label, inputs, footer	Body background, header/nav colours, padding, class selector	Fest events, schedule, rules and registration form
Electronic Store	Image, links, div containers, product cards, paragraphs, footer, inline/internal CSS	Class selectors, flexbox, gap, alignment, image dimensions, font styling	Store categories, product cards, prices, images and contact details

Common practical-exam execution checklist

• Keep each HTML file and its linked CSS file in the same folder unless the href specifies another path.

• Check spelling and capitalisation of every filename; filenames can be case-sensitive on some systems.

• Use a browser to open the HTML file and verify that the CSS is loading.

• Keep required image files in the expected folder.

• Check that every opening tag has the correct closing tag and that elements are properly nested.

• Remember: HTML builds the structure; CSS controls the presentation.

Final exam tip: If asked to explain an experiment, answer in this order: Introduction → Aim → HTML elements → CSS properties → Output → Result. Use the section headings in this document as your speaking sequence.